MREN 103 – Mechatronics and Robotics Design I
Mechatronics and Robotics Engineering Program
Queen's University, Kingston

Group #30
Laboratory #1 - P2 Project Stage 1

Student Names: Brady Cabral & Emily Vargas-Penaranda
Instructor Name: Prof. Surgenor
Date Report Submitted: March 29, 2026

Summary:
The objective of stage 1 was to put together all the acquired knowledge from previous labs into a full navigation cycle through the track shown in Fig 1. Reusing the line-following logic, in addition to the detection of a double black line in the middle of the track, the Minibot was able to successfully complete 2 continuous loops, also stopping within 30 cm or less from the wall placed in the middle and stops after completion of the loops without contact.

Hardware:
The Minibot is shown on the test area in Fig 2. below. The Minibot is comprised of a Sharp GP2D12 for range sensing mounted to the front of the robot, two light intensity sensors placed at the bottom of the Minibot to be used to track the line that is placed below the Minibot along with motors attached to wheels which will control movement of the robot. The Minibot is controlled by an ItsyBitsy. The track that is shown in Fig 1. and Fig 2. is comprised a whiteboard with black tape in a loop with a connected middle road and a wooden block for an obstacle.

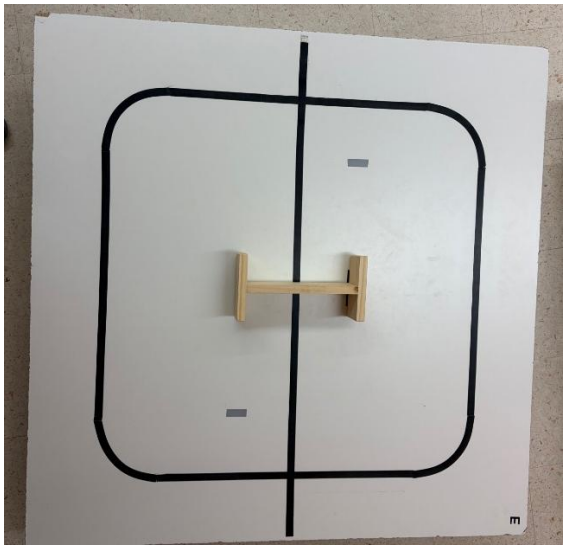

Fig 1. Blank track for Minibot to travel on

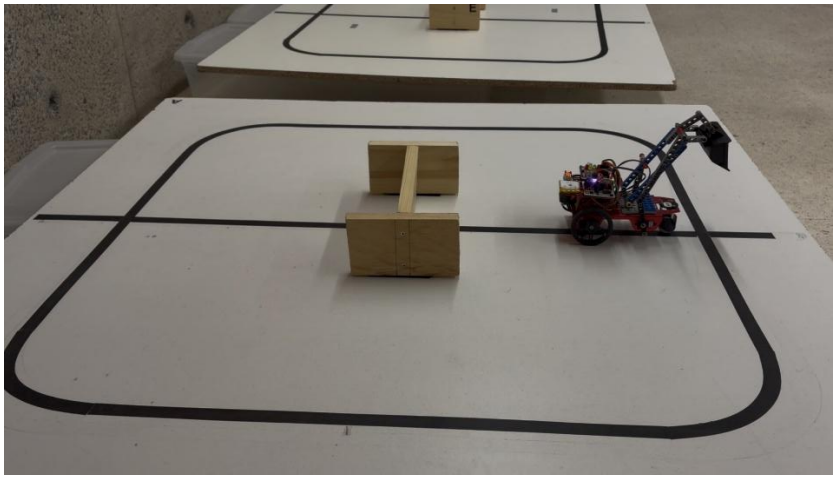

Fig 2. Minibot running on black track

Software:
A listing of the program ***Group30ProjectLoaderStage1*** used for the project is given in **Appendix A**. The program's objective is to follow the path outlined in the summary above. The setup function initializes all important global variables used by the program. The loop function's objective is to control the Minibot to follow the track in Fig 2. and complete stage 1 objectives outlined in the summary this was done by reading the values from the light intensity sensors if the left one was triggered it would turn right and if the right one was triggered it would turn left and if neither were triggered it would remain straight. This allowed the Minibot to follow the outer loop in Fig 2. but to turn into the inner line and detect the wall we added a condition such that if both left and right intensity triggers reached the threshold then the Minibot would turn a sharp ninety degrees clockwise. Then Minibot used the range sensor and if the reading came within a certain value (ie. Less than 30) it would stop the Minibot and do a sharp one hundred and eighty turn clockwise. A counter would also increase every time the bot turned one hundred and eighty degrees and if it surpassed a limit the next time the range sensor read a value below the threshold the Minibot would stop, this allowed us to stop the bot without contact as a piece of board would be placed at one of the corners of the outer track the bot would register that and stop. After every sharp turn the bot would enter a limbo like state where it would continue going forward until only its left intensity sensor was triggered then it would return to following normal conditions. This change was necessary because the sharp turns would place the bot off the line and due to the conditions set for normal travel in most situations the bot would then turn away from the line, which is disadvantageous. This allows for the bot to follow all requirements of stage 1.

Results*:*
The Minibot was able to achieve all requirements for Stage 1 evaluation though not without great difficulty. The greatest issue that we had when working on Stage 1 was the sharp ninety and one hundred and eighty degree turns. This was because initially we just had the robot stop the right motor and run the left one for a certain period of time however finding the appropriate time for turning was exceedingly difficult due to how unreliable the Minibot could be at time but the greater issue was that after the turns the Minibot would be extremely far off the line upwards of three centimeters, as shown in **Fig 3**. When the robot was this far off the line it would actively prevent itself from connecting with the line again due to the conditions set up to drive it. The one hundred and eighty degree turns faced the same challenge but was even more significant as the offset from the line was even greater. To fix this we changed two major things first we modified the turning such that now it spun both wheels in opposite directions for the sharp turns which significantly reduced the offset from the line. The next thing we added was a condition such that after a significant turn the robot would enter a limbo like state in which it would only go forward until just its left intensity sensor was triggered then it would return to normal which would allow the robot to properly reconnect with the black line.

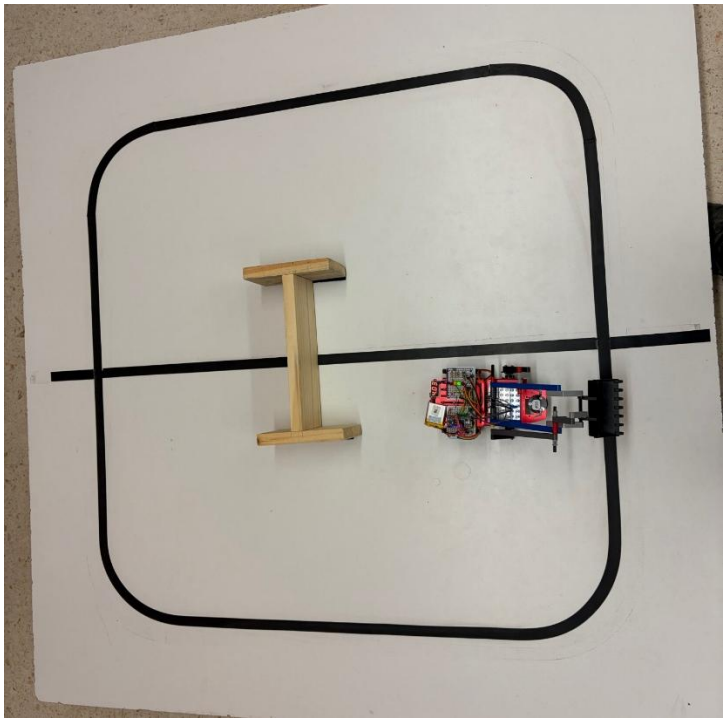

Fig 3. Minibot failing track by being too offset from track

Conclusions:
This project taught us how to reliably turn the robot 90 and 180 degrees whilst limiting how far it strays from the line and more importantly it showed us how to design the robot such that it can recover itself from failure thus improving reliability of the robot. The difficulty with the sharp ninety and one hundred eighty degree turns forced us to adapt and be better prepared for unreliable or inconsistent hardware.

Appendix A - Program Listing:

```
/*********************************************************
  Group30ProjectLoaderStage1

  Created by Brady C. & Emily V.

  Completes stage 1 of the loader project which
primarily entails following a black
  line trail
*********************************************************/

#define blcktp 2300 // 2000
#define DELTA 8
#define TURNDELTA 10
#define R_TURN_OFFSET 5
#define OFFSET 1
#define MAX_DIST 1250
#define Turn90Time 1270 // 1170
#define Turn180Time 1650 // 1550
#define PAUSETIME 1000

#include <Servo.h>
Servo leftWheel;
Servo rightWheel;

#define TurnTest false
bool moved = false;
bool just_turned = false;

// Pin Assignments
int RED = 10;          //red LED Pin
int GRN = 9;          //green LED Pin
int YLW = 5;          //yellow LED Pin
int BUTTON = 7;       //pushbutton Pin
int SHARP = A3;     // Sharp Sensor on Analog Pin 3
int MOTOR_R = 3;       // right motor signal pin
int MOTOR_L = 4;       // left motor signal pin

boolean loopbttn = true;

const int LSENSOR = A2; // Left Sensor on Analog Pin
1
const int RSENSOR = A1; // Right Sensor on Analog
Pin 2

const int stopPulse = 149;  // stop speed for motors
(default = 155)

const boolean PLOT = true;  //true = plot; false = print

int value = 0;
int mv_value = 0;

int stop_count = 0;

//global variables
int lvalue = 0;  //left sensor value
int rvalue = 0;  //right sensor value

// Set-up Routine
void setup() {

// Initialize led pins as outputs.
  pinMode(GRN, OUTPUT);
  pinMode(YLW, OUTPUT);
  pinMode(RED, OUTPUT);

// Initialize pins as inputs
  pinMode(BUTTON, INPUT);
  pinMode(SHARP, INPUT);

  pinMode(LSENSOR, INPUT);
  pinMode(RSENSOR, INPUT);

  leftWheel.attach(MOTOR_L);
  rightWheel.attach(MOTOR_R);

// Initialize serial printout
  Serial.begin(9600);    // default 9600

  runMotors(0,0);
}

// Main Routine
void loop() {

    while (loopbttn && digitalRead(BUTTON) == LOW) {
      digitalWrite(GRN, HIGH);
      delay(125);
      digitalWrite(GRN, LOW);
      delay(125);
    }
    loopbttn = false;
    moved = false;

    lvalue = analogRead(LSENSOR);
    rvalue = analogRead(RSENSOR);

    lvalue = map(lvalue,0,1023,0,3000);
    rvalue = map(rvalue,0,1023,0,3000);

    value = analogRead(SHARP);
    mv_value = map(value,0,1023,0,3300); //convert
AtoD count to millivolts
```

```
    if (TurnTest && !moved) {
      turnOnLED(GRN);
      runMotors(TURNDELTA, -TURNDELTA-
R_TURN_OFFSET);
      delay(Turn180Time);
      runMotors(0, 0);
      moved = true;
      loopbttn = true;
    }

    if (lvalue >= blcktp && rvalue < blcktp &&
just_turned) just_turned = false;

    if (mv_value < MAX_DIST && !TurnTest &&
!just_turned) {
      // black on both sensors
      if (rvalue > blcktp && lvalue > blcktp) {
        runMotors(0,0);
        delay(PAUSETIME);
        just_turned = true;
        turnOnLED(YLW);
        runMotors(TURNDELTA, -TURNDELTA-
OFFSET);
        delay(Turn90Time);
        runMotors(DELTA,DELTA-OFFSET);

      } // Black on both
      // Left side
      else if (rvalue > blcktp){
        turnOnLED(YLW);
        runMotors(DELTA, 0);

      }/* Right side */else if (lvalue > blcktp) {
        turnOnLED(RED);
        runMotors(0, DELTA);
      }/* Straddle */ else if (rvalue < blcktp && lvalue <
blcktp){

        turnOnLED(GRN);
        runMotors(DELTA,DELTA-OFFSET);
      }
      } else if (!TurnTest && mv_value >= MAX_DIST
&& stop_count < 4) { // make 180 turn spin both wheels
in opposite directions
        stop_count++;
        runMotors(0, 0);
        delay(PAUSETIME);
        just_turned = true;
        turnOnLED(YLW);
        runMotors(TURNDELTA, -TURNDELTA-
R_TURN_OFFSET);
        delay(Turn180Time);
        runMotors(DELTA,DELTA-OFFSET);
      } else if (!TurnTest && mv_value >= MAX_DIST) {
        runMotors(0, 0);
      }



}

//********** Functions (subroutines) ******************

// run robot wheels
void runMotors(int deltaL, int deltaR)
{
  int pulseL = (stopPulse + deltaL)*10;    //length of
pulse in microseconds
  int pulseR = (stopPulse + deltaR)*10;
  leftWheel.writeMicroseconds(pulseL);
  rightWheel.writeMicroseconds(pulseR);
}

// Turn on a single LED and turn others off
void turnOnLED(int COLOUR)
{
  digitalWrite(GRN, LOW);
  digitalWrite(YLW, LOW);
  digitalWrite(RED, LOW);
  digitalWrite(COLOUR, HIGH);
}
```